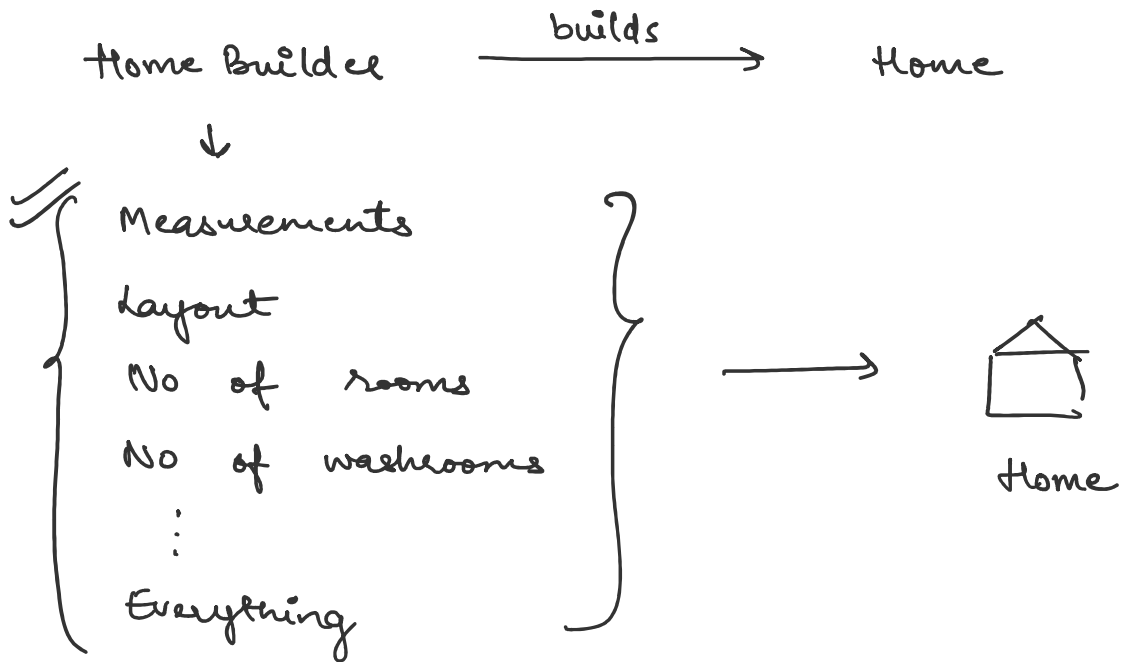
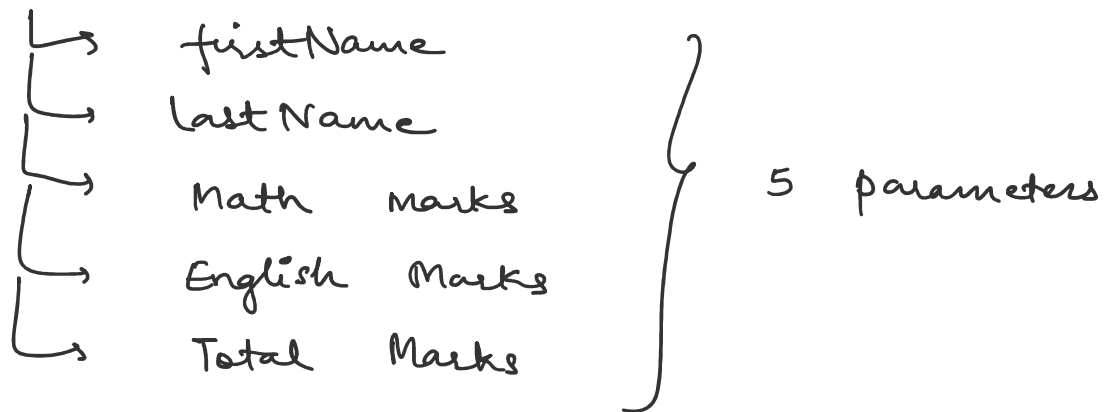


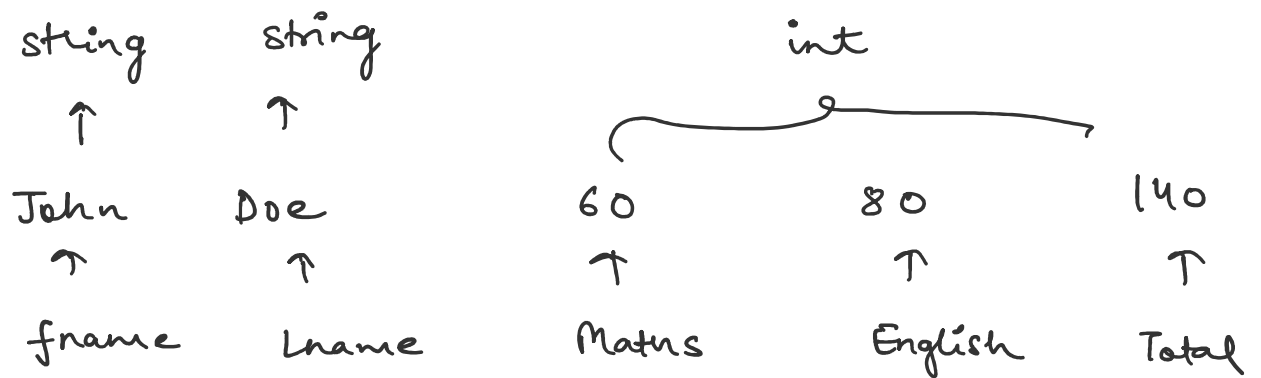
## Builder Design Pattern



Student



## Constructor of Student class -



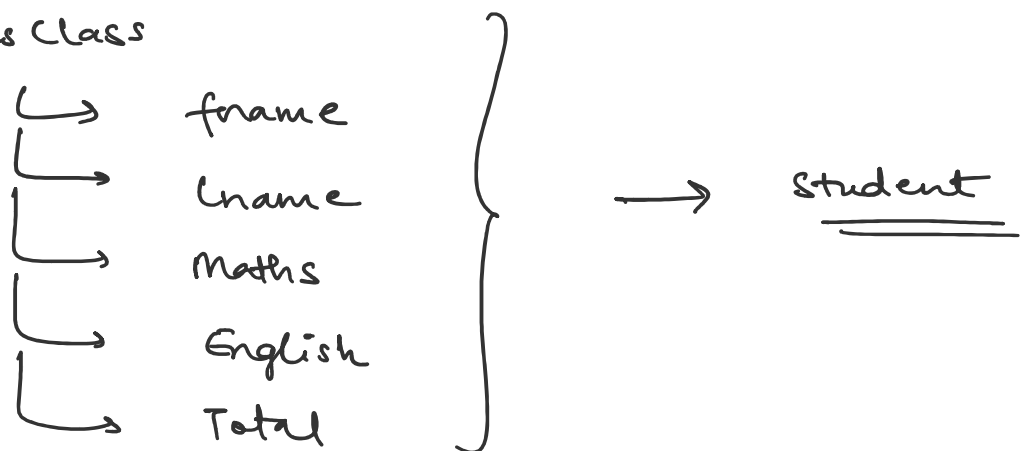
Student ( Doe , John , 140 , 60 , 80 )

Annotations below the arguments:

- ✓ above Doe, John, 140, 60, 80
- ↓ above x, x
- String brace under x, x
- int brace under 140, 60, 80
- ✓ above 140, 60, 80

Double checkmark (✓✓) below the int brace.

### Details Class



student (Details details) → Constructor  
{

fname = details.fname ;

lname = details.lname ;

maths = details.maths

,  
:  
:

}

4 + parameters → complex object  
↓  
class inject that      Bad design  
↓  
Good Design

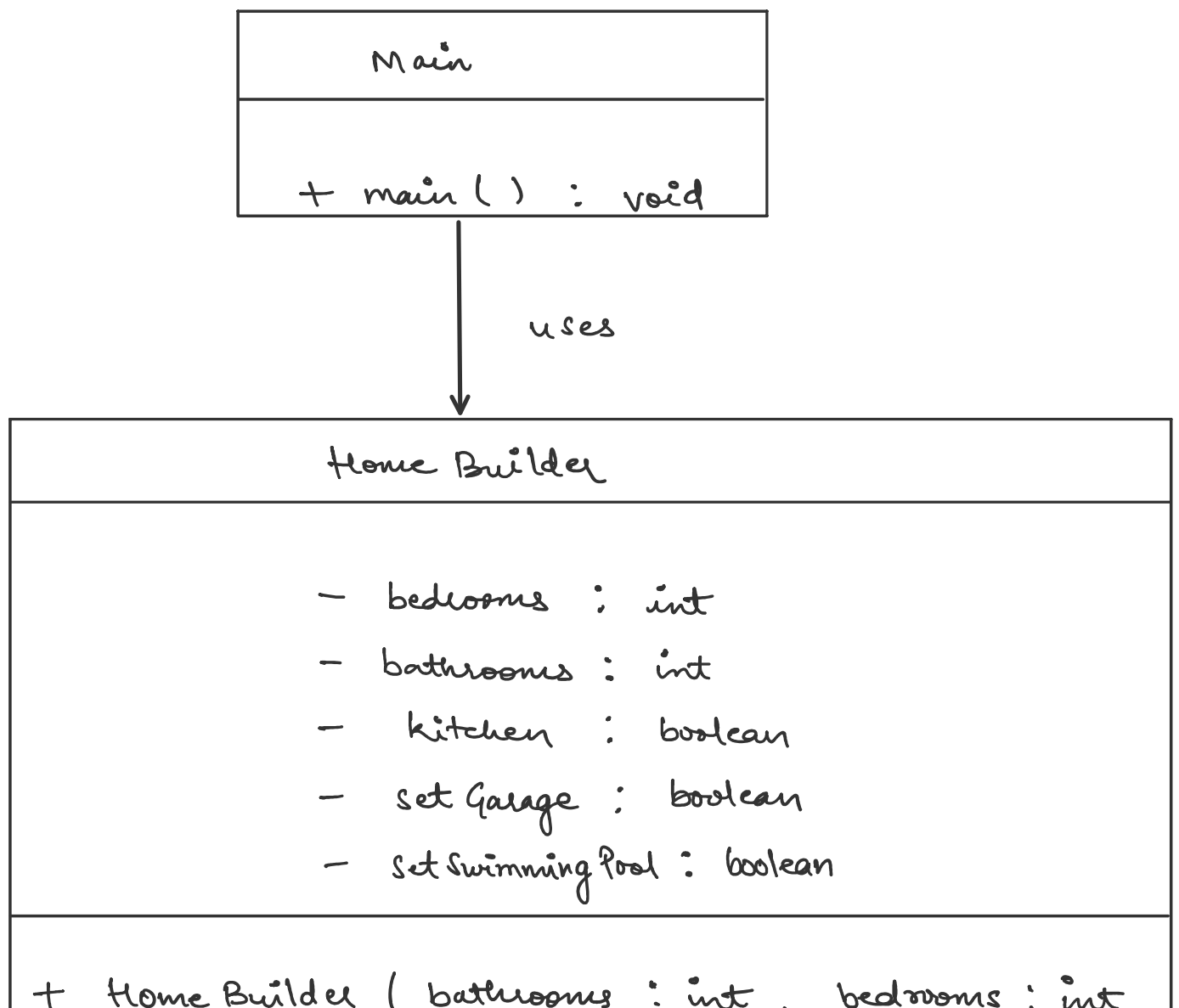
Home

|   |   |                 |   |           |   |
|---|---|-----------------|---|-----------|---|
| } | ↳ | 2 Bedrooms      | → | Mandatory | ✓ |
|   | ↳ | 3 Washrooms     | → | Mandatory | ✓ |
|   | ↳ | 1 swimming pool | → | optional  | ✓ |
|   | ↳ | 1 Kitchen       | → | Mandatory | ✓ |
|   | ↳ | 1 Garage        | → | optional  | ✓ |
|   | ↳ | Interior        | → | optional  | ✓ |



Apply Builder pattern

Class Diagram -



+ Home Builder ( bathrooms : int , bedrooms : int ,  
kitchen : boolean ) :: void

+ set Garage ( garage : boolean ) :: Home Builder

+ setSwimmingPool ( spool : boolean ) :: Home Builder

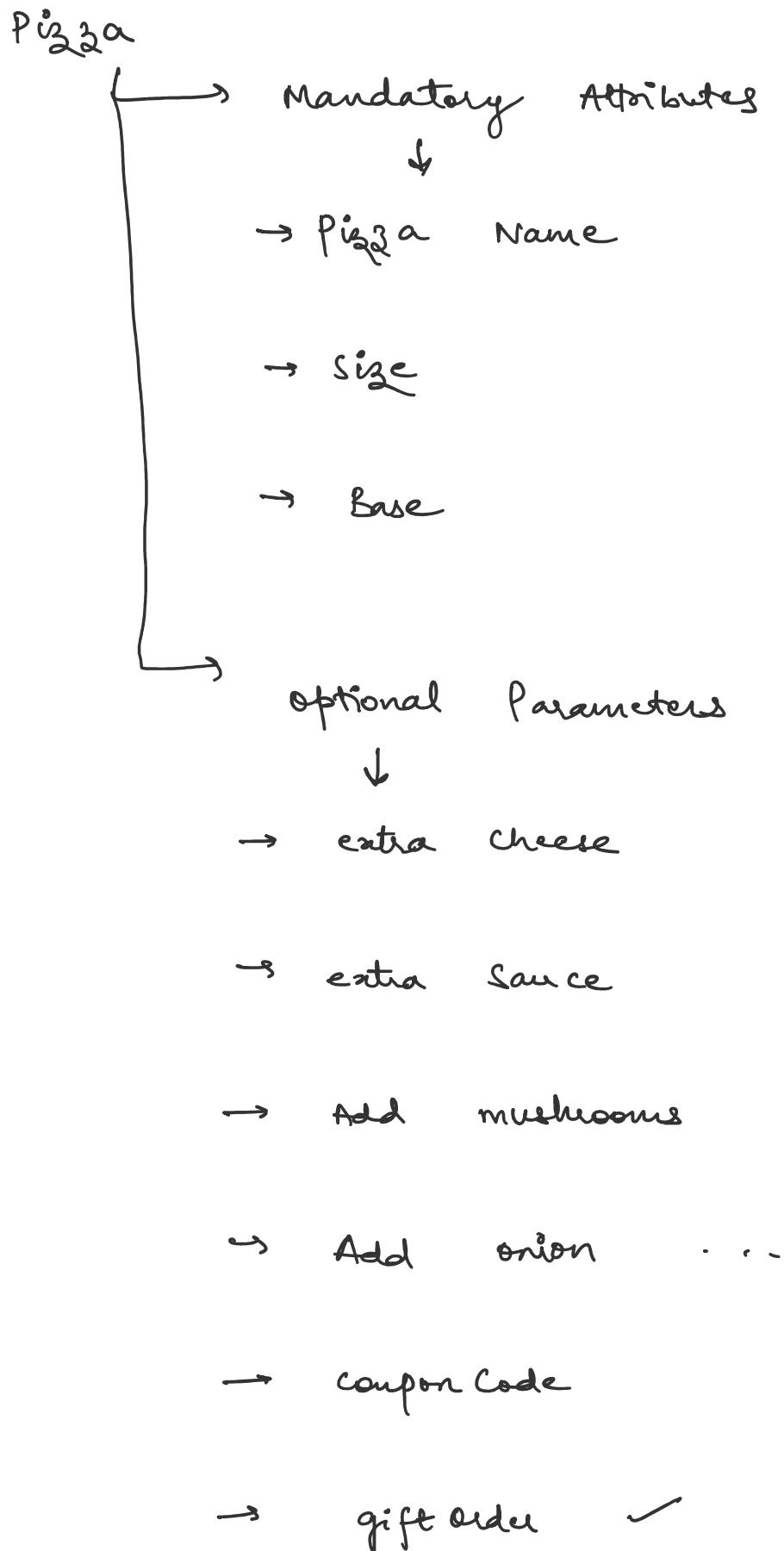
builds

Home

- bedrooms : int
- bathrooms : int
- kitchen : boolean
- set Garage : boolean
- setSwimmingPool : boolean

- Home ( home builder : Home Builder )
- + toString ( ) :: String

Another eg -



→ { special Instructions  
→ { contactless delivery }

Main (Client)

↓ uses

Pizza Order Builder =

↓ builds

Pizza

## Notes:

The **Builder Pattern** is used when you want to **construct a complex object step by step**, especially when :

- The object has **many attributes**.
- Some attributes are **mandatory** and some are **optional**.
- You want to **create different representations** of the object with the same construction process.

### Real-life analogy :

Imagine you're building a **house**.

- Mandatory parts: walls, roof, wiring, plumbing
- Optional parts: painting, interiors, wall art

You can't live in a house without walls and plumbing, but you **can** live without fancy interiors — that can be added later.

**Same with objects in code:** if you have a class with many attributes (some required, some optional), you use a **Builder** to create that object step by step.

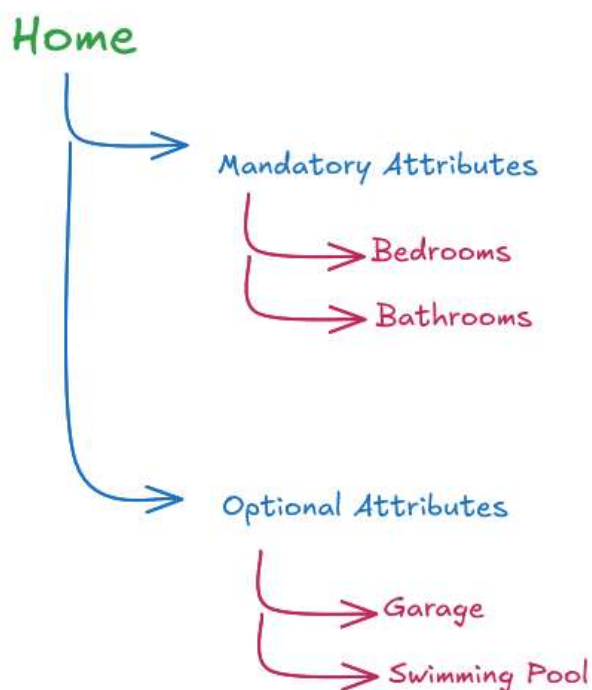
### When to Use Builder Pattern

- You are constructing a **complex object**.
- Object has **many attributes** (more than 4-5), some required, some optional.
- You want to **avoid multiple constructors** with many parameters.
- You want to **separate construction logic from the object itself**.
- You want to support **multiple versions/representations** of an object using the same build steps.



## Builder Components

1. **Product** → The complex object being built (e.g., Car, Home)
2. **Builder** → Static nested class inside Product that:
  - Takes **mandatory parameters** in its constructor
  - Provides **setter-like methods for optional attributes** (returning this for chaining)
  - Has a **build()** method that returns the constructed Product
3. **Client** → Uses the Builder directly to construct the object.



## Implementation of home example :

**1. Identify the Class with Many Parameters** - Pick a class (like Home) that has:

- **Mandatory fields** (e.g., bedrooms, bathrooms).
- **Optional fields** (e.g., garage, swimmingPool).
- A **private constructor** — so people can't directly create a Home without the builder.

**2. Make a HomeBuilder class inside the Home class** - This builder will help set all the values. In the builder :

- Declare all fields as private final : *private final int bedrooms;*
  - **private** - can't access it directly from outside
  - **final** - value can't change after it's set
- You give the required values in the **constructor**.
- Then you **optionally** set other values using methods like `setGarage(true)`. Each setter should return **this** for method chaining (fluent API) - It makes the code cleaner and easier to read.

### Without Chaining:

```
Home.HomeBuilder builder = new Home.HomeBuilder(3, 2);
builder.setGarage(true);
builder.setSwimmingPool(false);
Home home = builder.build();
```

### With Chaining (fluent API):

```
Home home = new Home.HomeBuilder(4, 3)
    .setGarage(true)
    .setSwimmingPool(false)
    .build();
```

- No public setters, because that would allow changes — and break immutability. Once built, the home can't be changed (i.e. it's

immutable)

- The constructor copies the values **from the builder**. Constructor should be **private**, and pass the Builder into it, so people **can't create a Home directly**, only the **builder** can call it. This makes sure that the object is fully ready before it's created and no one forgets to set a required value (like bedrooms).

```
private Home(HomeBuilder builder) {  
    this.bedrooms = builder.bedrooms;  
    this.bathrooms = builder.bathrooms;  
    this.garage = builder.garage;  
    this.swimmingPool = builder.swimmingPool;  
}
```

**3. Add a build() method** - This method creates and returns the real Home object.

**4. (Optional): Add a toString() method** - So when you print the Home, you get a nice description, not a weird memory address. For readable output.

*// CLIENT -> uses -> HOMEBUILDER -> creates/builds -> HOME*

```
public class Home {  
    // mandatory  
    private int bedrooms;  
    private int bathrooms;  
    private boolean kitchen;  
  
    // optional  
    private boolean garage;
```

```
private boolean swimmingPool;
```

```
// constructor
```

```
private Home(HomeBuilder builder) {  
    this.bedrooms = builder.bedrooms;  
    this.bathrooms = builder.bathrooms;  
    this.kitchen = builder.kitchen;  
    this.garage = builder.garage;  
    this.swimmingPool = builder.swimmingPool;  
}
```

```
@Override
```

```
public String toString(){  
    return "Home : bedrooms=" + bedrooms +  
        ", bathrooms=" + bathrooms +  
        ", kitchen=" + kitchen +  
        ", garage=" + garage +  
        ", swimmingPool=" + swimmingPool;  
}
```

```
public static class HomeBuilder{
```

```
    // mandatory
```

```
    private int bedrooms;  
    private int bathrooms;  
    private boolean kitchen;
```

```
    // optional
```

```
    private boolean garage;  
    private boolean swimmingPool;
```

```
    // constructor - setting mandatory fields
```

```
    public HomeBuilder(int bedrooms, int bathrooms, boolean  
kitchen) {
```

```

        // basic validation
        if (bedrooms <= 0 || bathrooms <= 0) {
            throw new IllegalArgumentException("bedrooms and
bathrooms must be positive");
        }

        this.bedrooms = bedrooms;
        this.bathrooms = bathrooms;
        this.kitchen = kitchen;
    }

    // optional parameters
    public HomeBuilder setSwimmingPool(boolean
swimmingPool) {
        this.swimmingPool = swimmingPool;
        return this; // enabling method chaining - returning this
lets you call another method right after
    }

    public HomeBuilder setGarage(boolean garage) {
        this.garage = garage;
        return this;
    }

    // build method - creating home instance
    public Home build() {
        return new Home(this);
    }
}

```

```

public class Main {
    public static void main(String[] args) {
        Home home1 = new Home.HomeBuilder(3, 2,true)
            .setSwimmingPool(true)
            .setGarage(false)
    }
}

```

```
.build();
```

```
System.out.println(home1); // Home : bedrooms=3,  
bathrooms=2, kitchen=true, garage=false, swimmingPool=true
```

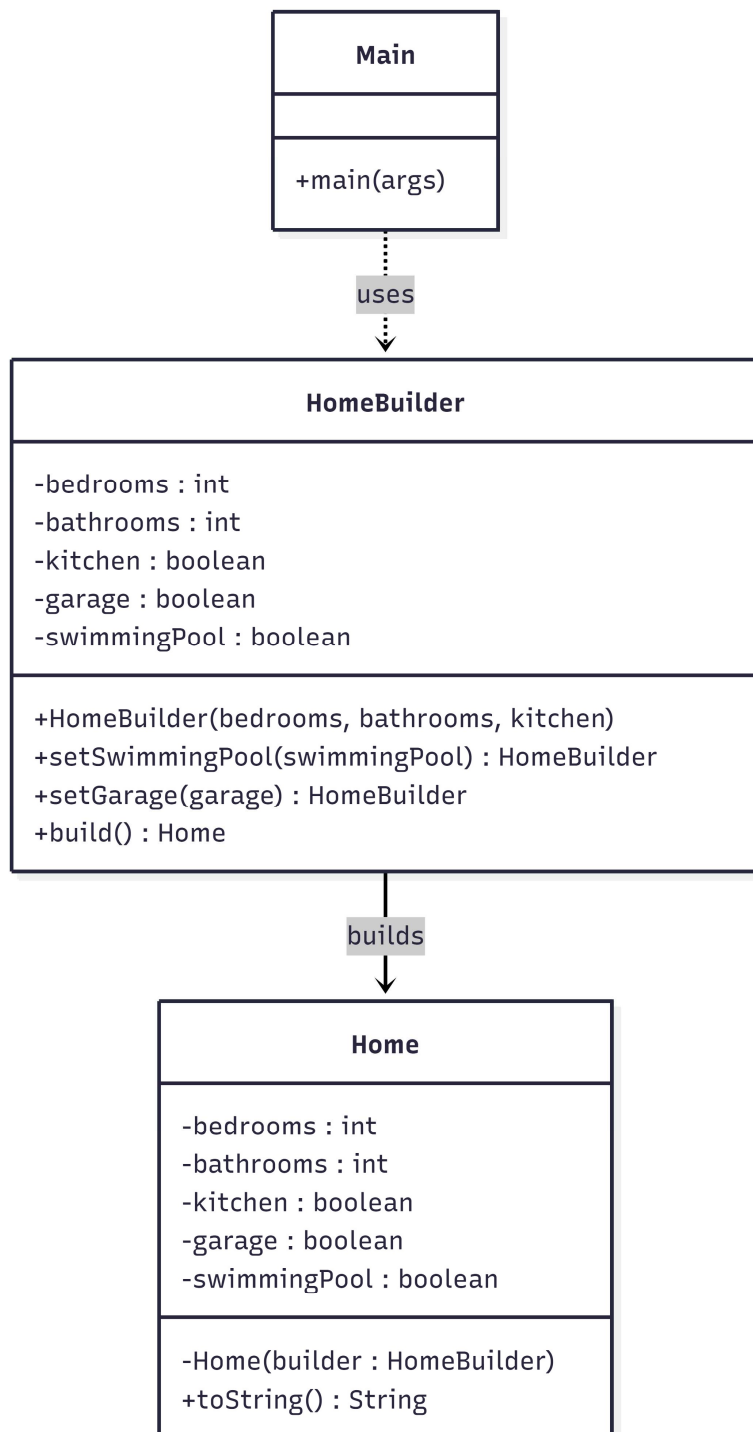
```
Home home2 = new Home.HomeBuilder(4, 4,true)  
    .setSwimmingPool(false)  
    .setGarage(false)  
    .build();
```

```
System.out.println(home2); // Home : bedrooms=4,  
bathrooms=4, kitchen=true, garage=false, swimmingPool=false  
    }  
}
```

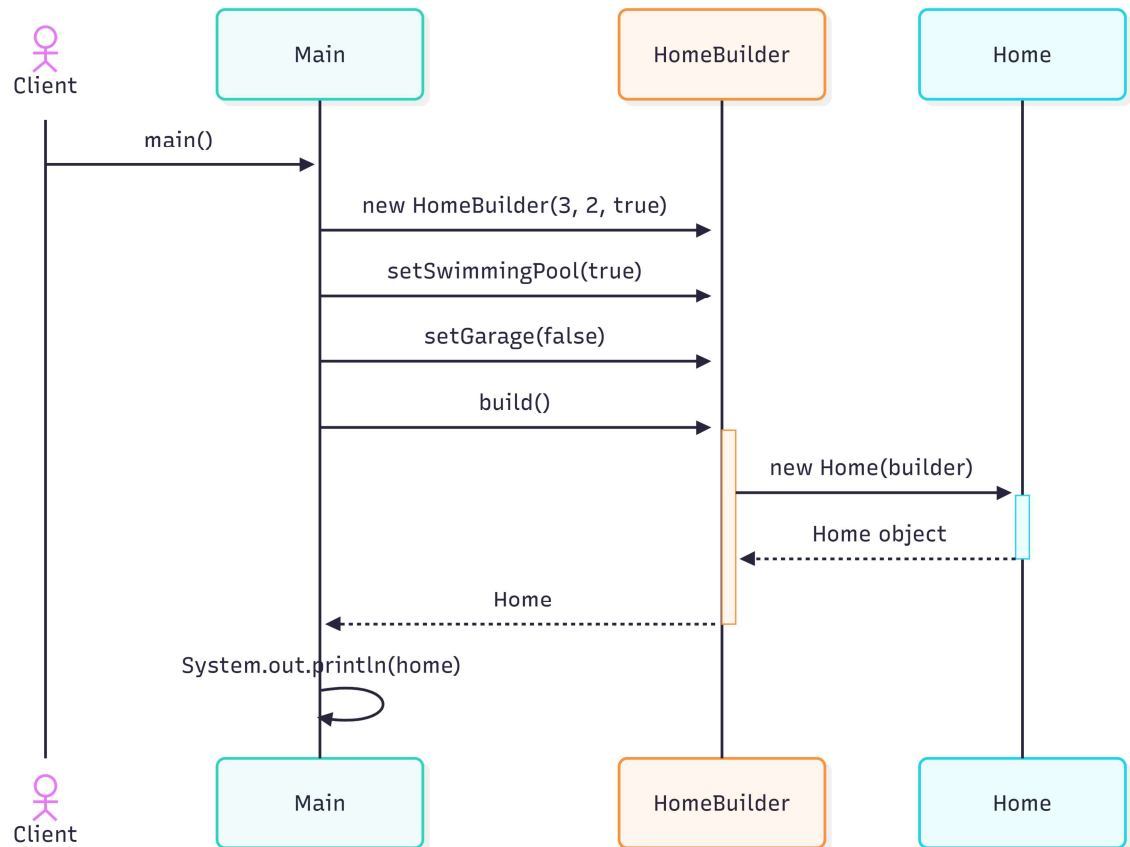
### Note:

Although the fields in **Home** and **HomeBuilder** look the same, they serve different purposes. The **Builder** temporarily stores all the values while the object is being constructed and configured. When **build()** is called, these values are copied into the Home object. The Home object then stores the final state permanently, while the builder is no longer needed. This separation allows the product to remain immutable and keeps the object creation process clean and flexible.

## Class Diagram

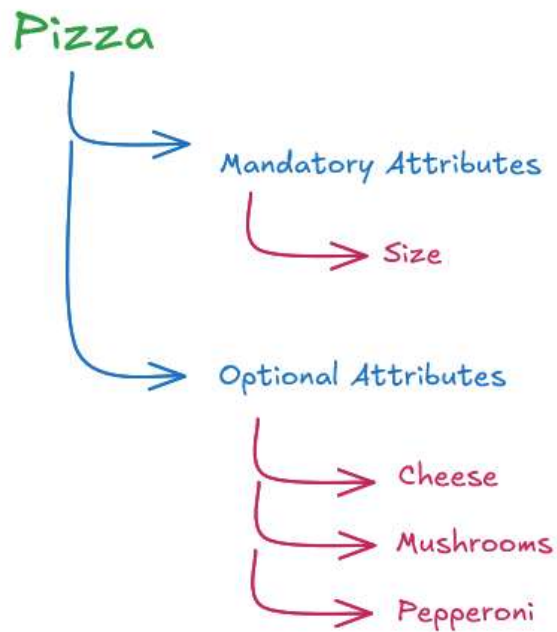


## Sequence Diagram





## Another Example



### Step 1. PizzaSize Enum

```
public enum PizzaSize {  
    SMALL,  
    MEDIUM,  
    LARGE  
}
```

### Step 2. CrustType Enum

```
public enum CrustType {  
    REGULAR,  
    THIN_CRUST,  
    CHEESE_BURST  
}
```

}

### Step 3. PizzaOrder

```
public class PizzaOrder {  
    // Mandatory Fields  
    private final String pizzaName;  
    private final PizzaSize size;  
  
    // Optional Fields  
    private final CrustType crustType;  
    private final boolean extraCheese;  
    private final boolean extraSauce;  
    private final int oreganoSachets;  
    private final int chilliFlakesSachets;  
    private final String couponCode;  
    private final boolean giftWrap;  
    private final boolean contactlessDelivery;  
    private final String specialInstructions;  
  
    private PizzaOrder(PizzaOrderBuilder builder) {  
        this.pizzaName = builder.pizzaName;  
        this.size = builder.size;  
        this.crustType = builder.crustType;  
        this.extraCheese = builder.extraCheese;  
        this.extraSauce = builder.extraSauce;  
        this.oreganoSachets = builder.oreganoSachets;  
        this.chilliFlakesSachets = builder.chilliFlakesSachets;  
        this.couponCode = builder.couponCode;  
        this.giftWrap = builder.giftWrap;  
        this.contactlessDelivery = builder.contactlessDelivery;  
        this.specialInstructions = builder.specialInstructions;  
    }  
}
```

@Override

```
public String toString() {  
    return "PizzaOrder" +  
        "\n pizzaName=" + pizzaName + "\" +  
        ",\n size=" + size +  
        ",\n crustType=" + crustType +  
        ",\n extraCheese=" + extraCheese +  
        ",\n extraSauce=" + extraSauce +  
        ",\n oreganoSachets=" + oreganoSachets +  
        ",\n chilliFlakesSachets=" + chilliFlakesSachets +  
        ",\n couponCode=" + couponCode + "\" +  
        ",\n giftWrap=" + giftWrap +  
        ",\n contactlessDelivery=" + contactlessDelivery +  
        ",\n specialInstructions=" + specialInstructions + "\";  
}
```

```
public static class PizzaOrderBuilder {
```

```
    // Mandatory
```

```
    private final String pizzaName;  
    private final PizzaSize size;
```

```
    // Optional
```

```
    private CrustType crustType = CrustType.REGULAR;  
    private boolean extraCheese;  
    private boolean extraSauce;  
    private int oreganoSachets = 1;  
    private int chilliFlakesSachets = 1;  
    private String couponCode = "";  
    private boolean giftWrap;  
    private boolean contactlessDelivery;  
    private String specialInstructions = "";
```

```
    public PizzaOrderBuilder(String pizzaName, PizzaSize size)  
{  
  
    if (pizzaName == null || pizzaName.isBlank()) {  
        throw new IllegalArgumentException("Pizza name is  
required.");  
    }  
}
```

```
    }

    if (size == null) {
        throw new IllegalArgumentException("Pizza size is
required.");
    }

    this.pizzaName = pizzaName;
    this.size = size;
}

public PizzaOrderBuilder crustType(CrustType crustType) {
    if (crustType != null) {
        this.crustType = crustType;
    }
    return this;
}

public PizzaOrderBuilder extraCheese(boolean value) {
    this.extraCheese = value;
    return this;
}

public PizzaOrderBuilder extraSauce(boolean value) {
    this.extraSauce = value;
    return this;
}

public PizzaOrderBuilder oreganoSachets(int count) {
    if (count < 0) {
        throw new IllegalArgumentException("Oregano sachets
cannot be negative.");
    }
    this.oreganoSachets = count;
    return this;
}

public PizzaOrderBuilder chilliFlakesSachets(int count) {
```

```
        if (count < 0) {  
            throw new IllegalArgumentException("Chilli flakes  
sachets cannot be negative.");  
        }  
        this.chilliFlakesSachets = count;  
        return this;  
    }
```

```
    public PizzaOrderBuilder couponCode(String couponCode) {  
        if (couponCode != null) {  
            this.couponCode = couponCode.trim();  
        }  
        return this;  
    }
```

```
    public PizzaOrderBuilder giftWrap(boolean value) {  
        this.giftWrap = value;  
        return this;  
    }
```

```
    public PizzaOrderBuilder contactlessDelivery(boolean value)  
{  
        this.contactlessDelivery = value;  
        return this;  
    }
```

```
    public PizzaOrderBuilder specialInstructions(String  
instructions) {  
        if (instructions != null) {  
            this.specialInstructions = instructions;  
        }  
        return this;  
    }
```

```
    public PizzaOrder build() {  
  
        if (giftWrap &&  
            crustType == CrustType.CHEESE_BURST) {
```

```

        throw new IllegalStateException(
            "Gift wrapping is not available for Cheese Burst
pizzas.");
    }

    if (oreganoSachets > 10) {
        throw new IllegalStateException(
            "Maximum 10 oregano sachets are allowed.");
    }

    if (extraSauce &&
        size == PizzaSize.SMALL) {
        throw new IllegalStateException(
            "Extra sauce is not available for Small pizzas.");
    }

    return new PizzaOrder(this);
}
}
}

```

#### Step 4. Main

```

public class Main {
    public static void main(String[] args) {
        PizzaOrder order1 =
            new PizzaOrder.PizzaOrderBuilder(
                "Farmhouse",
                PizzaSize.LARGE)
                .crustType(CrustType.THIN_CRUST)
                .extraCheese(true)
                .extraSauce(true)
                .oreganoSachets(3)
                .chilliFlakesSachets(2)

```

```
.couponCode("SAVE200")
.giftWrap(true)
.specialInstructions("Ring the bell once.")
.build();
```

```
System.out.println(order1);
```

```
System.out.println("-----");
```

```
PizzaOrder order2 =
    new PizzaOrder.PizzaOrderBuilder(
        "Veggie Delight",
        PizzaSize.MEDIUM)
        .extraCheese(true)
        .contactlessDelivery(true)
        .specialInstructions("Leave at the doorstep.")
        .build();
```

```
System.out.println(order2);
```

```
System.out.println("-----");
```

```
PizzaOrder order3 =
    new PizzaOrder.PizzaOrderBuilder(
        "Margherita",
        PizzaSize.SMALL)
        .build();
```

```
System.out.println(order3);
```

```
    }
}
```

```
/* *
```

*Output :*  
*PizzaOrder*

*pizzaName='Farmhouse',  
size=LARGE,  
crustType=THIN\_CRUST,  
extraCheese=true,  
extraSauce=true,  
oreganoSachets=3,  
chilliFlakesSachets=2,  
couponCode='SAVE200',  
giftWrap=true,  
contactlessDelivery=false,  
specialInstructions='Ring the bell once.'*

---

#### *PizzaOrder*

*pizzaName='Veggie Delight',  
size=MEDIUM,  
crustType=REGULAR,  
extraCheese=true,  
extraSauce=false,  
oreganoSachets=1,  
chilliFlakesSachets=1,  
couponCode="",  
giftWrap=false,  
contactlessDelivery=true,  
specialInstructions='Leave at the doorstep.'*

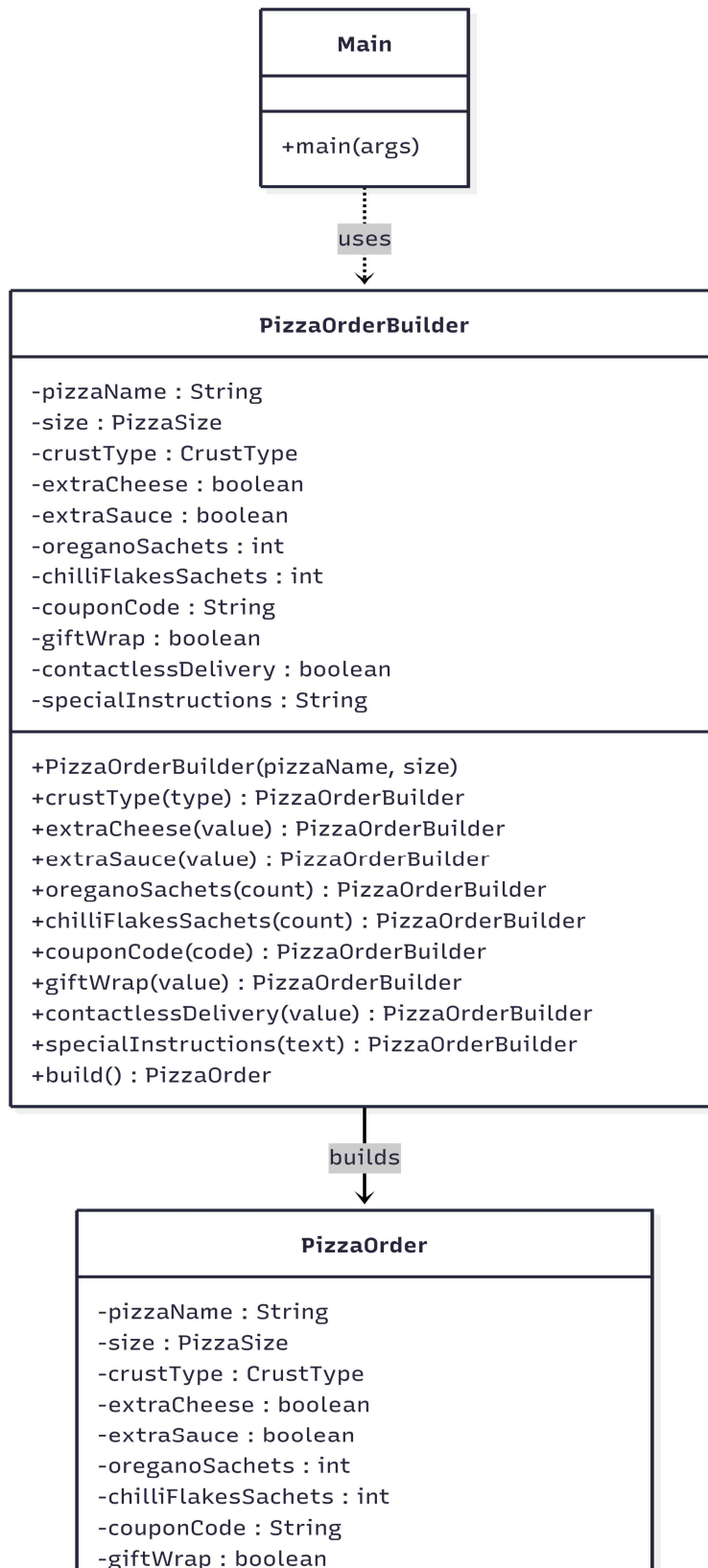
---

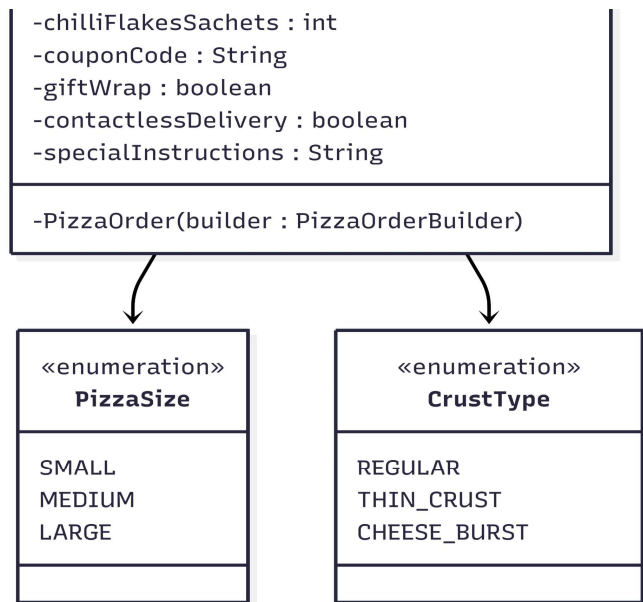
#### *PizzaOrder*

*pizzaName='Margherita',  
size=SMALL,  
crustType=REGULAR,  
extraCheese=false,  
extraSauce=false,  
oreganoSachets=1,  
chilliFlakesSachets=1,  
couponCode="",  
giftWrap=false,  
contactlessDelivery=false,  
specialInstructions=""*

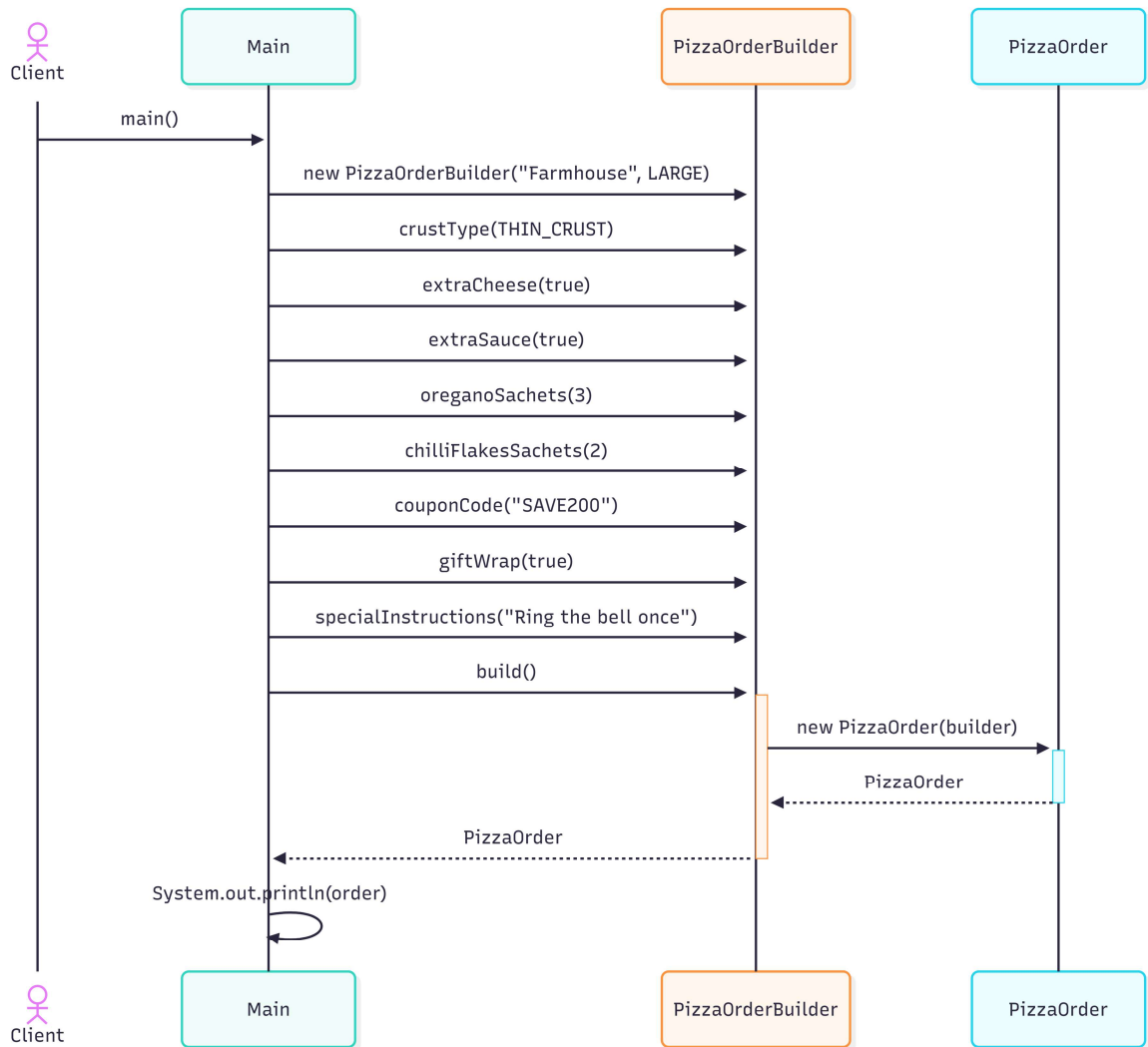


## Class Diagram





# Sequence Diagram



| <b>Aspect</b>            | <b>Builder</b>                                 | <b>Factory</b>                   | <b>Abstract Factory</b>                         |
|--------------------------|------------------------------------------------|----------------------------------|-------------------------------------------------|
| <b>Purpose</b>           | Step-by-step object creation                   | Single method to create object   | Create families of related objects              |
| <b>Object Complexity</b> | Complex objects with many attributes           | Simple objects                   | Group of related objects                        |
| <b>Control</b>           | Fine-grained control over construction process | Less control (one-shot creation) | Multiple factories for related products         |
| <b>Focus</b>             | How to build                                   | What to build                    | Which family to build                           |
| <b>Analogy</b>           | Building a house step by step                  | Ordering a pizza                 | Ordering a combo meal (pizza + drink + dessert) |